

Complemento Práctico: Modelado de Relaciones 1 a 1

De Atributos Primitivos a Objetos Colaborativos

En esta actividad práctica dejamos de ver a los objetos como islas de datos primitivos (int, String) y comenzamos a conectarlos. Esta transición representa un salto fundamental en la programación orientada a objetos: pasar de estructuras simples a sistemas colaborativos donde los objetos se comunican y dependen unos de otros.

Analizaremos cómo el código Java cambia drásticamente dependiendo de si la relación es una simple asociación, una agregación flexible o una composición rígida. El foco estará en la implementación técnica: ¿Dónde hago el new? ¿Cómo evito bucles infinitos en relaciones bidireccionales? ¿Qué pasa con la memoria cuando elimino un objeto?

Estas preguntas no son triviales. Las respuestas determinan la arquitectura de tu aplicación, su mantenibilidad y su comportamiento en tiempo de ejecución. Dominar estas relaciones te permitirá diseñar sistemas robustos y escalables.



Contenido

Atributos de tipo Objeto

Transición de primitivos a referencias de objetos complejos

Representación correcta en UML

Diagramación profesional de relaciones entre clases

Asociación Bidireccional

Prevención del StackOverflow en referencias mutuas

Ciclos de vida

Diferencias críticas entre Agregación y Composición

Más allá de los Primitivos

El primer desafío es mental: un Motor no cabe en un String. Si intentamos modelar un sistema real, los atributos de una clase suelen ser instancias de otras clases. Esta comprensión marca la diferencia entre programar con tipos básicos y construir sistemas orientados a objetos verdaderos.

En el video vimos el caso del Auto. En lugar de describir el motor con texto, creamos una clase Motor para encapsular su propia lógica (cilindrada, tipo, número). Esto no solo organiza mejor el código, sino que permite que el Motor tenga comportamiento propio: puede encenderse, regularse, reportar su estado.

El Auto tiene una variable de instancia que es una **referencia** a un objeto Motor. Esta referencia es fundamental: no guardamos una copia del motor, sino la dirección de memoria donde vive el objeto Motor.

Implementación en Java

```
public class Auto {  
    // No usamos String,  
    // usamos la clase creada  
    private Motor motor;  
  
    // Constructor recibe  
    // el objeto ya creado  
    public Auto(Motor motor) {  
        this.motor = motor;  
    }  
}
```

Representación en UML

Un error común al pasar a UML es escribir el atributo motor: Motor dentro de la caja de la clase Auto. Este enfoque genera diagramas saturados y pierde la esencia del modelado orientado a objetos: las relaciones entre entidades.



Dibujar ambas clases
Auto y Motor como entidades separadas e independientes



Conectar con línea
La flecha indica navegabilidad: quién tiene referencia a quién

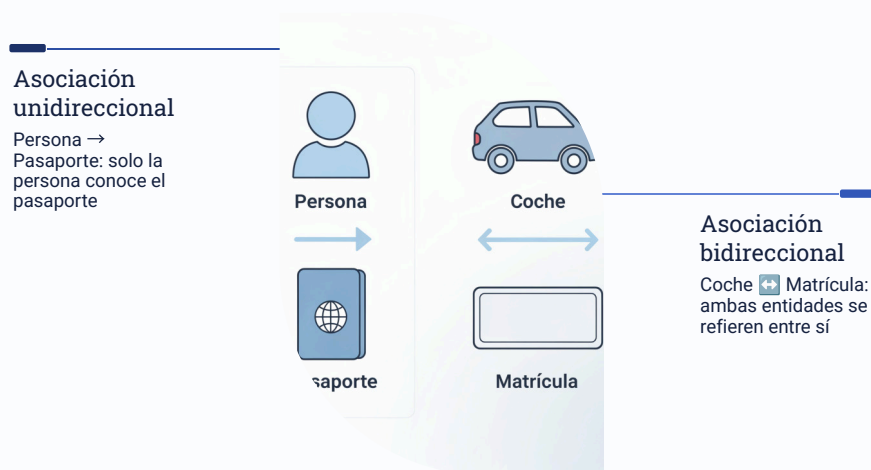


Indicar cardinalidad
Los números especifican cuántos objetos participan en la relación

Al sacar el atributo de la caja y convertirlo en una relación gráfica, el diagrama comunica mejor la arquitectura del sistema y reduce el ruido visual dentro de la clase. Esta técnica se vuelve crítica cuando trabajas con sistemas de 20, 50 o 100 clases interconectadas.

El Desafío de la Bidireccionalidad

La **Asociación Unidireccional** es simple y segura: Persona tiene Pasaporte. La persona conoce su pasaporte, pero el pasaporte no sabe de quién es. Esta simplicidad evita complicaciones, pero a veces la lógica de negocio requiere navegación en ambos sentidos.



El problema surge en la **Bidireccionalidad**, donde ambos objetos se referencian mutuamente. Imagina un Coche y su Matrícula: el coche conoce su matrícula, y la matrícula debe saber a qué coche pertenece. Parece lógico, pero la implementación incorrecta genera un desastre.

El Problema del Bucle Infinito

Si `Coche.setMatricula()` llama a `Matricula.setCoche()`, y este vuelve a llamar a `Coche.setMatricula()`, generamos un **StackOverflowError**. La pila de llamadas crece infinitamente hasta agotar la memoria.

Solución a la Bidireccionalidad

La Validación Preventiva

Debemos validar si la asignación ya existe antes de invocar al otro lado. Esta comprobación rompe el ciclo recursivo y mantiene la consistencia de ambas referencias.

La clave está en verificar dos condiciones: que el objeto no sea null y que la referencia inversa no apunte ya a nosotros. Solo si ambas condiciones se cumplen, actualizamos el otro lado de la relación.

Este patrón se repite en todas las relaciones bidireccionales.

Memorizarlo te ahorrará horas de depuración buscando StackOverflows misteriosos.

Observa cómo la condición `matricula.getCoche() != this` es fundamental. Preguntamos: "¿Esta matrícula ya apunta a mí?". Si la respuesta es sí, no hacemos nada. Si es no, completamos la relación bidireccional. Simple, elegante y seguro.

```
// En clase Coche
public void setMatricula(
    Matricula matricula) {

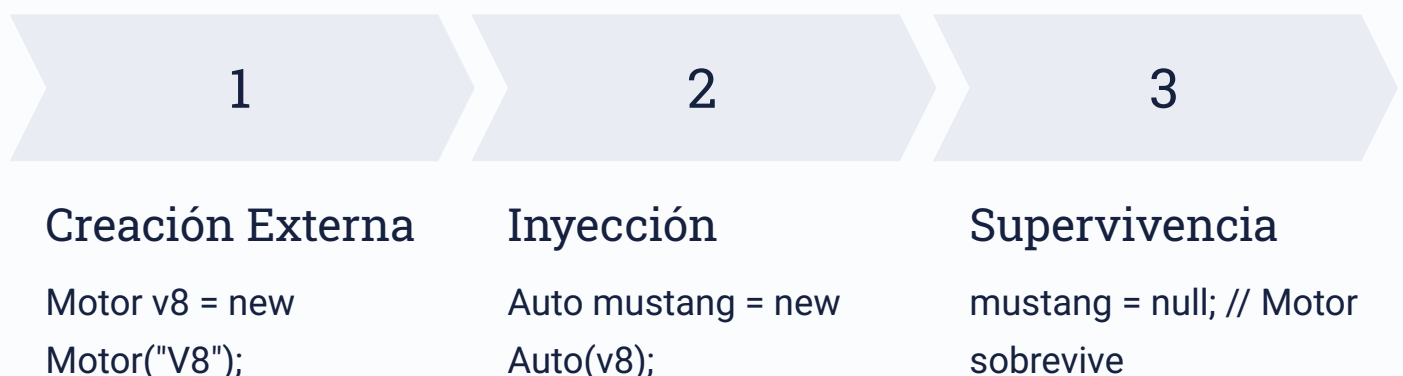
    this.matricula = matricula;

    // Validación clave para
    // romper recursión infinita
    if (matricula != null &&
        matricula.getCoche() != this) {
        matricula.setCoche(this);
    }
}
```

Agregación: Vidas Independientes

La agregación representa una relación "Tiene-Un" donde la parte vive fuera del todo. Es como un estudiante que tiene una laptop: la laptop existe independientemente del estudiante. Si el estudiante se gradúa y deja la universidad, la laptop sigue existiendo y puede ser usada por otra persona.

Clave de Implementación: El objeto contenido se crea **afuera** y se pasa por parámetro (usualmente en el constructor). Este detalle técnico tiene consecuencias profundas en el ciclo de vida de los objetos.



Si el objeto Auto deja de existir (se hace null), el objeto Motor **sigue existiendo** en memoria porque fue instanciado externamente. Esto es extremadamente útil cuando múltiples objetos pueden compartir la misma parte, o cuando la parte tiene un ciclo de vida más largo que el contenedor.

Composición: Ciclo de Vida Dependiente

La composición representa una relación fuerte "Parte-De" donde la parte no tiene sentido sin el todo. Es como el corazón de una persona: no existe independientemente. Si la persona deja de existir, el corazón también desaparece del contexto útil.

Clave de Implementación: El objeto contenido se crea **dentro** del constructor de la clase contenedora. Nadie más tiene acceso directo a esa instancia. Es una creación privada e interna.

```
public class Libro {  
    private Portada portada;  
  
    public Libro(String titulo,  
                String datosPortada) {  
        this.titulo = titulo;  
  
        // Creación INTERNA  
        // Nadie más tiene referencia  
        this.portada =  
            new Portada(datosPortada);  
    }  
}
```

Garbage Collection

Si el objeto 'libro' pasa a ser null, el Garbage Collector de Java se lleva también a la 'portada' porque no hay otras referencias apuntando a ella.

Esta es la muerte conjunta: ambos objetos están tan íntimamente ligados que su existencia es mutuamente dependiente. La portada fue creada para ese libro específico y no tiene vida fuera de él.

Cuadro de Decisiones de Diseño

Para la actividad práctica, utiliza esta guía rápida al escribir tu código. Estas decisiones de diseño no son arbitrarias: responden a la naturaleza de las relaciones en el dominio del problema que estás modelando.



Asociación

new: Externo (Main)

Independencia: Total. Solo se conocen

UML: Flecha simple

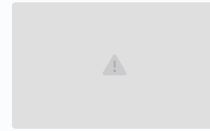


Agregación

new: Externo (Main)

Independencia: Alta. La parte sobrevive

UML: Rombo Blanco



Composición

new: Interno (Constructor)

Independencia: Nula. Muerte conjunta

UML: Rombo Negro



Tip de experto

Comienza siempre con relaciones simples (Asociación/Agregación). Usa Composición solo cuando la lógica de negocio exija estrictamente que la parte no pueda existir sola. Ejemplos clásicos: Factura y Líneas de Detalle, Casa y Habitaciones, Documento y Párrafos.

Próximos Pasos

Ahora que comprendes las bases teóricas y técnicas de las relaciones 1 a 1, es momento de aplicar estos conceptos en código real. La actividad práctica te desafiará a implementar cada tipo de relación, enfrentándote a decisiones de diseño auténticas.

Practica con ejemplos reales

Implementa casos como Estudiante-Carnet, Empleado-OficinaAsignada, Producto-Empaque

Diagrama antes de codificar

Diseña en UML primero para visualizar relaciones y evitar refactorizaciones

Prueba la destrucción

Experimenta haciendo null a objetos para verificar ciclos de vida correctos

Recuerda: dominar estas relaciones no es memorizar sintaxis, es desarrollar intuición sobre cómo los objetos colaboran en sistemas complejos. Con cada ejercicio, estarás más cerca de pensar naturalmente en términos de objetos, responsabilidades y relaciones. ¡Éxito en tu práctica!